VOLUME 8

COMPLETE VOLUME — CHAPTERS 8.1–8.7

Security

Securing the Platform — and Closing Every Deferred Security Item Since Volume 1

Document Title	Volume 8 — Security (Combined)
Volume Reference	Volume 8
Chapters Included	8.1 through 8.7 (7 chapters)
Version	1.0
Status	Draft — Pending Approval (8.6/8.7 additionally require legal review before publication)
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Governed By	Volume 0 — Project Foundation; grounded in Volumes 1, 4, 5, and 6

Table of Contents

This volume secures the platform and resolves every security-related item earlier volumes explicitly deferred: encryption decisions (NFR-SEC-02/03), API rate limiting, IAM/secrets management, authentication hardening, a draft privacy policy, and concrete data retention windows (BR-23, FR-META-14, NFR-META-04, NFR-SEC-04).

Chapter	Title
8.1	Security Architecture
8.2	Encryption
8.3	API Security
8.4	AWS Security
8.5	Authentication Security
8.6	Privacy Policy
8.7	Data Retention

VOLUME 8 — SECURITY

CHAPTER 8.1

Security Architecture

The Defense-in-Depth Posture Across Everything Built So Far

Document Title	Security Architecture
Chapter Reference	8.1
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Purpose

Security was never deferred entirely to this volume — Volume 4 built authentication and authorization, Volume 6 built secure token storage, Volume 3 fixed offline-first data handling. This chapter is the synthesis: it names every layer of defense already in place, and this volume's remaining chapters (8.2–8.7) close the specific gaps those earlier volumes explicitly left open.

2. Defense Layers, As Already Built

Layer	Mechanism	Established In
Identity	Firebase Authentication; ID token verified server-side on every request	Volume 4, Chapter 4.7
Authorization	Role + org/assignment scoping enforced in shared middleware, never client-trusted	Volume 4, Chapter 4.8
Transport	HTTPS-only for the API; presigned, time-limited HTTPS URLs for S3	Volume 4, Chapter 4.10
Credential storage (device)	Hardware-backed Keychain/Keystore, never plain text	Volume 6, Chapter 6.7
Network boundary	Aurora inside a private VPC subnet, reachable only from Lambda	Volume 4, Chapter 4.9
Least privilege	Per-function IAM roles; per-role/org query scoping	Volume 4, Chapters 4.8/4.9

3. Threat Model — Key Scenarios Considered

Lost or stolen Collector device: mitigated by OS-level device encryption (Chapter 8.2), short-lived local exposure of unconfirmed chunks (BR-08, Volume 5), and secure token storage (Chapter 6.7) rather than a persisted password.

Compromised or malicious Collector account: authorization scoping (BR-19, Volume 4.8) limits blast radius to that Collector's own assigned Tasks — never another Collector's or another org's data.

Compromised Admin account: scoped to a single org (BR-20); Section 8.5 adds further hardening around Admin-specific actions.

Man-in-the-middle on a public network: mitigated entirely by TLS everywhere (Section 2) — no endpoint in this system accepts plaintext HTTP.

Backend infrastructure compromise: mitigated by network segmentation (VPC), least-privilege IAM (Chapter 8.4), and encryption at rest (Chapter 8.2) limiting what a breach of one component exposes.

4. What This Volume Still Resolves

- 8.2 — encryption at rest, closing NFR-SEC-02/03's "evaluated in Volume 8.2" reference.
- 8.3 — API rate limiting, closing Volume 4, Chapter 4.6's deferral.

- 8.4 — IAM policy detail and secret management, closing Volume 4, Chapters 4.7/4.9's deferrals.
- 8.5 — authentication hardening beyond the basic flow in Volume 4.7.
- 8.6/8.7 — privacy and retention, closing NFR-SEC-04, BR-23, FR-META-14, and every "Volume 8.7" reference across Volumes 1, 4, and 5.

VOLUME 8 — SECURITY

CHAPTER 8.2

Encryption

Resolving NFR-SEC-02/03's 'Evaluated in Volume 8.2'

Document Title	Encryption
Chapter Reference	8.2
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Encryption in Transit

- Every API call (Volume 4, Chapter 4.6) and every S3 upload (presigned HTTPS URLs, Chapter 4.10) uses TLS 1.2 or higher — no endpoint in this system accepts plaintext HTTP, enforced at the API Gateway and S3 bucket policy level, not left to client discipline.

2. Encryption at Rest — Backend

Store	Mechanism
S3 (chunk storage)	SSE-S3 by default (Volume 4, Chapter 4.10); upgradeable to SSE-KMS with a customer-managed key — this chapter's Section 4 makes that upgrade decision.
Aurora Serverless v2	Encryption at rest enabled at cluster creation (AWS default for new Aurora clusters) — covers chunk_metadata, users, projects, tasks, and every other backend table.
Secrets (Firebase service account, DB credentials)	AWS Secrets Manager, encrypted with a KMS key — detailed in Chapter 8.4.

3. ADR-011 — Local Video Chunk Encryption at Rest

ADR-011 — Local (On-Device) Chunk Encryption at Rest

Status: Accepted

Context:

NFR-SEC-03 requires locally stored chunks to be 'treated as sensitive until upload is confirmed.' The open question this ADR resolves: is OS-level app-sandbox protection sufficient, or does the app need its own additional file-level encryption layer?

Options Considered:

- App-level AES encryption of every chunk file, with a device-held key — strongest guarantee, but adds real CPU/battery cost to a 10-minute continuous video write (Volume 5, Chapter 5.4) and meaningful implementation complexity (key management, encrypt-on-write/decrypt-on-read-for-upload).
- Rely solely on OS-level protection — modern Android (File-Based Encryption, default since Android 10) and iOS (Data Protection, default Class) already encrypt app-sandboxed storage at rest, and no other app can read this app's sandbox without root/jailbreak.

Decision:

Chosen: Rely on OS-level app-sandbox encryption as the baseline for MVP; do not add a custom app-level encryption layer on top.

Consequences:

- No additional battery/CPU cost is added to the Recording Pipeline (Volume 5, Chapter 5.4), keeping its performance budget (NFR-META-01) unaffected.
- The exposure window is already short by construction — BR-08 (Volume 1) deletes a chunk the moment upload is confirmed, so the OS-encrypted-at-rest window is typically minutes to hours, not indefinite.

- This decision is explicitly revisited if a future client contract or regulatory review (Chapter 8.6) requires app-level encryption as a compliance checkbox regardless of the OS's own protection — flagged here as a known, deliberate reopening trigger, not a closed question forever.

4. S3 SSE-S3 vs. SSE-KMS — Decision

For MVP: SSE-S3 (Volume 4, Chapter 4.10's default) is sufficient — it already encrypts every object at rest with AWS-managed keys.

Upgrade trigger: move to SSE-KMS with a customer-managed key only if a specific client contract requires demonstrable key-level access control/audit trail (KMS key usage is independently logged in CloudTrail, Chapter 8.4) — not adopted preemptively, since it adds operational key-rotation overhead with no immediate benefit.

5. What This Chapter Defers

- IAM policy detail controlling who/what can even reach these encrypted resources — Chapter 8.4.

VOLUME 8 — SECURITY

CHAPTER 8.3

API Security

Resolving Volume 4, Chapter 4.6's Deferred Rate Limiting and Quota Policy

Document Title	API Security
Chapter Reference	8.3
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Rate Limiting

Endpoint Category	Limit	Rationale
Auth verify (POST /v1/auth/verify)	20 requests/minute/user	Generous enough for normal token-refresh cycles; blocks brute-force-style hammering.
Chunk registration/metadata (POST /v1/sessions/.../chunks, /metadata)	120 requests/minute/user	A Collector could legitimately finalize several chunks in quick succession across concurrent sessions (FR-UPL-04) — sized with headroom above realistic peak.
Admin read endpoints (projects/tasks/sessions/metadata reads)	300 requests/minute/user	Dashboard-style polling is read-heavy; generous since reads are cheap and non-mutating.
Admin write endpoints (create/edit Project/Task, assign Collector)	30 requests/minute/user	Deliberately tighter — these are low-frequency, human-driven actions; a burst here is more likely abuse than legitimate use.

Enforced at API Gateway (Volume 4, Chapter 4.9) via usage plans per authenticated user, not per IP alone — a shared office network's multiple Collectors are never incorrectly throttled as one client.

2. Input Validation

- Every Lambda handler validates its request body against a strict schema before touching the database — rejecting unexpected fields rather than silently ignoring them, which prevents a client from smuggling unexpected data into a write path.
- UUIDs (chunk_id, session_id, etc.) are validated as well-formed before any query — preventing malformed input from reaching SQL, on top of Aurora's own parameterized queries (never string-concatenated SQL, per Volume 3, Chapter 3.7's standards extended here to backend code).

3. Replay & Duplicate Protection

Already substantially solved by design: Volume 5, Chapter 5.14's deterministic chunk keys and Volume 4's idempotent status-transition endpoints (Chapter 4.10, Section 2) mean a replayed request either no-ops safely or produces an identical result — this chapter adds no new mechanism here, only confirms the existing design already satisfies API-security replay concerns.

4. Dependency & Supply-Chain Security

- Backend Node.js dependencies follow the same review process Volume 3, Chapter 3.8 established for the mobile app — license check, maintenance-activity check, and now additionally an automated vulnerability scan (npm audit or an equivalent SCA tool) gating CI (Volume 7, Chapter 7.13).
- Any dependency with a published critical CVE blocks deployment until patched or explicitly risk-accepted by the project owner — no silent ship-with-known-vulnerability path.

5. What This Chapter Does Not Cover

- Network-level protections (WAF, DDoS mitigation) — covered in Chapter 8.4 (AWS Security) since they're infrastructure configuration, not API-contract-level concerns.

VOLUME 8 — SECURITY

CHAPTER 8.4

AWS Security

Resolving the IAM and Secret-Management Items Deferred From Volume 4

Document Title	AWS Security
Chapter Reference	8.4
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. IAM Policy Detail Per Lambda Function

Volume 4, Chapter 4.9 named the principle (least-privilege, per-function roles); this chapter fixes the actual policy shape:

Function	Allowed Actions	Denied by Omission
chunk-registration	s3:PutObject (presigned URL generation only — no direct object access), rds-data:ExecuteStatement on chunks/sessions tables	Cannot read arbitrary S3 objects; cannot touch users/projects/tasks tables.
metadata-write	rds-data:ExecuteStatement on chunk_metadata only	No S3 permissions at all — mirrors Volume 4, Chapter 4.9's original example exactly.
projects-tasks	rds-data:ExecuteStatement on projects/tasks/task_assignments	No S3 permissions; cannot touch chunk_metadata.
auth-verify	No database write access — read-only lookup on users by firebase_uid	Cannot modify any table; a compromised token-verification path can't be leveraged into a data-write path.

2. Secrets Management

AWS Secrets Manager, not environment variables: the Firebase Admin SDK service account key (Volume 4, Chapter 4.7's deferred item) and the Aurora connection credentials are both stored in Secrets Manager, referenced by ARN from each Lambda's configuration — never baked into a Lambda's plaintext environment variables, which would be visible to anyone with read access to the function's configuration.

Rotation: Aurora credentials rotate automatically via Secrets Manager's native RDS rotation support; the Firebase service account key is rotated manually on a scheduled cadence (annually, or immediately on suspected compromise).

3. Network & Perimeter

- AWS WAF attached to API Gateway, with managed rule sets for common injection/exploit patterns, complementing Chapter 8.3's application-level input validation rather than replacing it.
- Security groups on the Aurora VPC allow inbound traffic only from the Lambda functions' own security group — not from any other CIDR range, including other AWS resources in the same account.
- S3 bucket public access is blocked at the account level (AWS's Block Public Access setting), not just at the individual bucket policy level, so a future misconfiguration can't accidentally expose it.

4. Logging & Audit

- CloudTrail is enabled account-wide, capturing every API call against AWS resources (who generated a presigned URL, who touched Secrets Manager) — a separate, infrastructure-level audit trail from the application-level audit_log table (Volume 4, Chapter 4.4).
- GuardDuty is enabled for automated threat detection (anomalous API calls, credential exfiltration patterns) across the account.

5. What This Chapter Defers

- Application-level authentication hardening (password policy, session revocation) — Chapter 8.5.

VOLUME 8 — SECURITY

CHAPTER 8.5

Authentication Security

Hardening Beyond Volume 4, Chapter 4.7's Base Flow

Document Title	Authentication Security
Chapter Reference	8.5
Version	1.0
Status	Draft — Pending Approval
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

1. Password Policy

Enforced via Firebase Authentication's own configurable password policy (minimum length 8, requiring a mix of character classes) rather than a custom implementation — no reason to reinvent what the already-chosen provider (Volume 4, Chapter 4.1) offers natively.

2. Brute-Force & Account Lockout

- Firebase Authentication's built-in rate limiting on sign-in attempts is relied on as the first line of defense; Chapter 8.3's auth-verify endpoint rate limit (20/min/user) is the second, independent line on the backend side.
- Repeated failed attempts against a single account trigger Firebase's own progressive delay — no custom lockout logic is built, avoiding a second, potentially inconsistent lockout policy running alongside the provider's own.

3. Token Lifecycle

Short-lived ID tokens (~1 hour), longer-lived refresh tokens: already the flow in Volume 4, Chapter 4.7 — this chapter adds the explicit security rationale: a stolen ID token has a naturally bounded window of usefulness, while the refresh token (the more sensitive artifact) never leaves the device's secure storage (Volume 6, Chapter 6.7).

Revocation on role change: removing a Collector's Task assignment (FR-ADM-04) does not need to revoke their token — Volume 4, Chapter 4.8's authorization scoping is re-evaluated on every request from the current task_assignments state, so access is effectively revoked on the very next API call regardless of token validity, without needing a separate token-blacklist mechanism.

Full account deactivation: for a Collector or Admin leaving the organization entirely, their Firebase account is disabled (not deleted, preserving audit_log integrity, Volume 4 Chapter 4.4) — a disabled Firebase account fails token verification immediately on its next refresh attempt.

4. Admin-Specific Hardening

Because an Admin account has broader blast radius than a Collector's (able to reassign/remove Collectors across an entire org, Volume 4 Chapter 4.8), destructive Admin actions already require an explicit confirmation step at the UX layer (Volume 2, Chapter 2.9, Section 2) — this chapter adds that a future Phase 2 hardening step (mandatory re-authentication before an Admin's most destructive actions, or optional MFA) is a reasonable next increment, though not required for MVP given the org-scoping already in place (BR-20).

5. What This Chapter Defers

- Multi-factor authentication as a built feature — not MVP scope (Volume 1, Chapter 1.10); noted here as a natural Phase 2 candidate for Admin accounts specifically, given their broader authorization scope.

VOLUME 8 — SECURITY

CHAPTER 8.6

Privacy Policy

What Data Is Collected, Why, and Under What Commitments

Document Title	Privacy Policy
Chapter Reference	8.6
Version	1.0
Status	Draft — Pending Approval — Legal Review Required Before Publication
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026

0. Important Note

This chapter drafts the substantive content and commitments a privacy policy needs, grounded in exactly what the system actually collects (Volume 1, Chapter 1.3's FR-META fields). It is not legal advice, and this draft is not fit to publish as-is: data protection obligations (GDPR, CCPA, and worker-location/monitoring laws in particular) vary by jurisdiction and by whether Collectors are employees or contractors. A qualified privacy/employment attorney should review this chapter — especially Sections 2 and 4 — before anything based on it is published or operationalized.

1. Data Collected

Category	Specific Data	Source
Account identity	Name, email, role (Admin/Collector)	Volume 4, Chapter 4.4 — users table
Device information	Device model, OS version, app version	FR-META-04
Location	GPS coordinates at the moment of chunk capture, if location permission is granted	FR-META-05 — the single most sensitive field this system collects
Work-context content	The recorded video/audio itself, plus Collector-authored notes/tags	Core product function; FR-META-07
Technical/usage data	Battery level, network type, upload/checklist events (Volume 9, Chapter 9.3)	FR-META-05, Chapter 9.3

2. Why It's Collected

- Identity and device data: to attribute footage to the correct Collector/device for BR-21/22's integrity and audit requirements.
- Location: to record where a Task was performed — a legitimate, disclosed business purpose (field data collection), never used for continuous or off-task tracking; the app only captures location at the moment of chunk finalization (Volume 5, Chapter 5.7), not continuously in the background outside an active recording session.
- Video/audio content: the core deliverable of the product — the footage itself is the work product being collected on behalf of Human Archive's clients.

3. Third-Party Processors

Processor	Role
Amazon Web Services	Hosts the backend API, database, and video storage (Volume 4).
Google Firebase	Authentication, push notifications, crash reporting (Volume 4, Chapter 4.1; Volume 6, Chapter 6.9).

Both are named explicitly rather than described vaguely as "service providers" — a common, reasonable transparency expectation and, in several jurisdictions, a legal requirement to name sub-processors.

4. People Incidentally Captured on Camera

The consideration this system must not gloss over: because recording is egocentric/wide-angle field footage (BR-01/02), bystanders — coworkers, members of the public, other people at a Task site — may be incidentally captured without their own direct consent to this system.

This is an operational/legal question for Human Archive, not something the app's engineering can resolve alone: recommended next steps (again, subject to actual legal review) include on-site signage or verbal notice at recording locations, contractual terms with client organizations addressing bystander footage, and a defined process for handling a bystander's request to be excluded or have footage reviewed.

5. Collector/Admin Rights (Draft Commitments)

- Access: a Collector may request a copy of their own account data and any metadata (not raw footage, which belongs to the commissioning client/org) tied to their account.
- Correction: a Collector may request correction of inaccurate account information (name, email) — never of system-generated metadata (BR-22), which is immutable by design.
- Retention and deletion: governed by Chapter 8.7, immediately following — this policy's retention commitments must match that chapter's technical implementation exactly, not describe a different timeline.

6. What This Chapter Defers

- The exact retention windows referenced in Section 5 — Chapter 8.7.
- Store-listing privacy disclosures (Play Data Safety / App Store privacy labels) that must be consistent with this chapter — Volume 10, Chapters 10.3/10.4.

VOLUME 8 — SECURITY

CHAPTER 8.7

Data Retention

Resolving Every 'Volume 8.7' Reference Since Volume 1

Document Title	Data Retention

Chapter Reference	8.7
Version	1.0
Status	Draft — Pending Approval — Legal Review Recommended
Owner	Faisal — Human Archive
Prepared With	Claude (Cowork)
Date	05 August 2026
Resolves	BR-23, FR-META-14, NFR-META-04, NFR-SEC-04 (Volume 1); Volume 4, Chapters 4.5/4.10; Volume 5, Chapter 5.15

0. Important Note

As with Chapter 8.6, specific retention durations below are a proposed technical/operational default, not a legal determination — actual required or permissible retention periods depend on the client contract governing each Project and on jurisdiction-specific law. This chapter should be reviewed against both before being treated as final policy.

1. Retention Windows

Data	Default Retention	After Which
Raw video chunks (S3)	24 months from capture (configurable per Project/client contract)	Transitions per Volume 4, Chapter 4.10's lifecycle (Standard → Standard-IA → Glacier) throughout, then deleted from S3 entirely unless a legal hold (Section 3) applies.
Chunk metadata (chunk_metadata)	7 years from capture, or the raw video's deletion plus 5 years, whichever is longer	Deliberately far outlives the raw footage (BR-23, FR-META-14) — metadata is lightweight and remains the only durable evidence of what was captured, when, by whom, once the video itself is gone.
Account data (users)	Retained while the account is active; soft-deleted (disabled, not erased) on offboarding, per Chapter 8.5	audit_log entries referencing the account are retained regardless, per Section 2.
audit_log	7 years, matching metadata's window	Supports the same long-tail audit/QA need as metadata retention.
Crash reports / logs (Volume 6, Chapter 6.9; Volume 9, Chapter 9.2)	90 days	Short window reflecting their debugging purpose rather than any evidentiary one.

2. Why Metadata Outlives Video by Design

This is the concrete number behind what Volume 4 and Volume 5 already described qualitatively: NFR-META-04 requires metadata to 'remain queryable through the retention window' — this chapter is what actually defines that window's length (7 years, Section 1), long enough to cover typical contractual/audit needs for a client's field-data-collection engagement, while the multi-hundred-MB video files (the far more expensive thing to store) age out much sooner.

3. Legal Hold

- Any Project, Session, or specific chunk can be flagged with a legal_hold_at timestamp (a small addition to Volume 4, Chapter 4.4's schema) — while set, it is excluded from every automated deletion/lifecycle job in Sections 1 and 4, regardless of age.
- Only an Admin (or a future Super Admin, Volume 1 Chapter 1.6) can set or clear a legal hold, and doing so is itself an audit_log entry — a hold can never be silently applied or removed.

4. Deletion Mechanics

S3 lifecycle transitions/expiration: native S3 lifecycle rules (Volume 4, Chapter 4.10) handle the storage-class transitions and eventual object expiration automatically — no custom deletion Lambda needed for the video files themselves.

Metadata/audit_log purge: a scheduled Lambda (EventBridge-triggered, monthly) identifies chunk_metadata/audit_log rows past their 7-year window and without an active legal hold, and purges them in a batch — this is the one piece of custom deletion logic this system runs, since Aurora has no native equivalent to S3 lifecycle rules.

5. Consistency With Chapter 8.6

The Privacy Policy's retention commitments (Chapter 8.6, Section 5) must state exactly these windows, not a different or vaguer number — any future change to this chapter's retention periods requires updating Chapter 8.6 in the same revision, not independently.